Data analysis&visualisation

Presented by Shivam Sharma

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import pymysql
import numpy as np

conn = pymysql.connect(
    host = "localhost",
    user = "root",
    password = "12@sshivam",
    database = "ecommerce"
)
cur = conn.cursor()
```

2. Count the number of orders placed in 2017.

```
query = f'select count(order_id) from orders where year(order_purchase_timestamp) = 2017'
cur.execute(query)
data = cur.fetchall()
"Total orders placed in 2017 are ",data[0][0]
```

[3] Python

```
('Total orders placed in 2017 are ', 45101)
```

3. Find the total sales per category.

```
query = '''select upper(products.product_category) as category, round(sum(payments.payment_value),2) as sales
    from products join orderitems on products.product_id = orderitems.product_id join payments on
    payments.order_id = orderitems.order_id group by category'''
cur.execute(query)
data = cur.fetchall()
data

df = pd.DataFrame(data, columns=["category", "sales"])
df
```

[4] Python

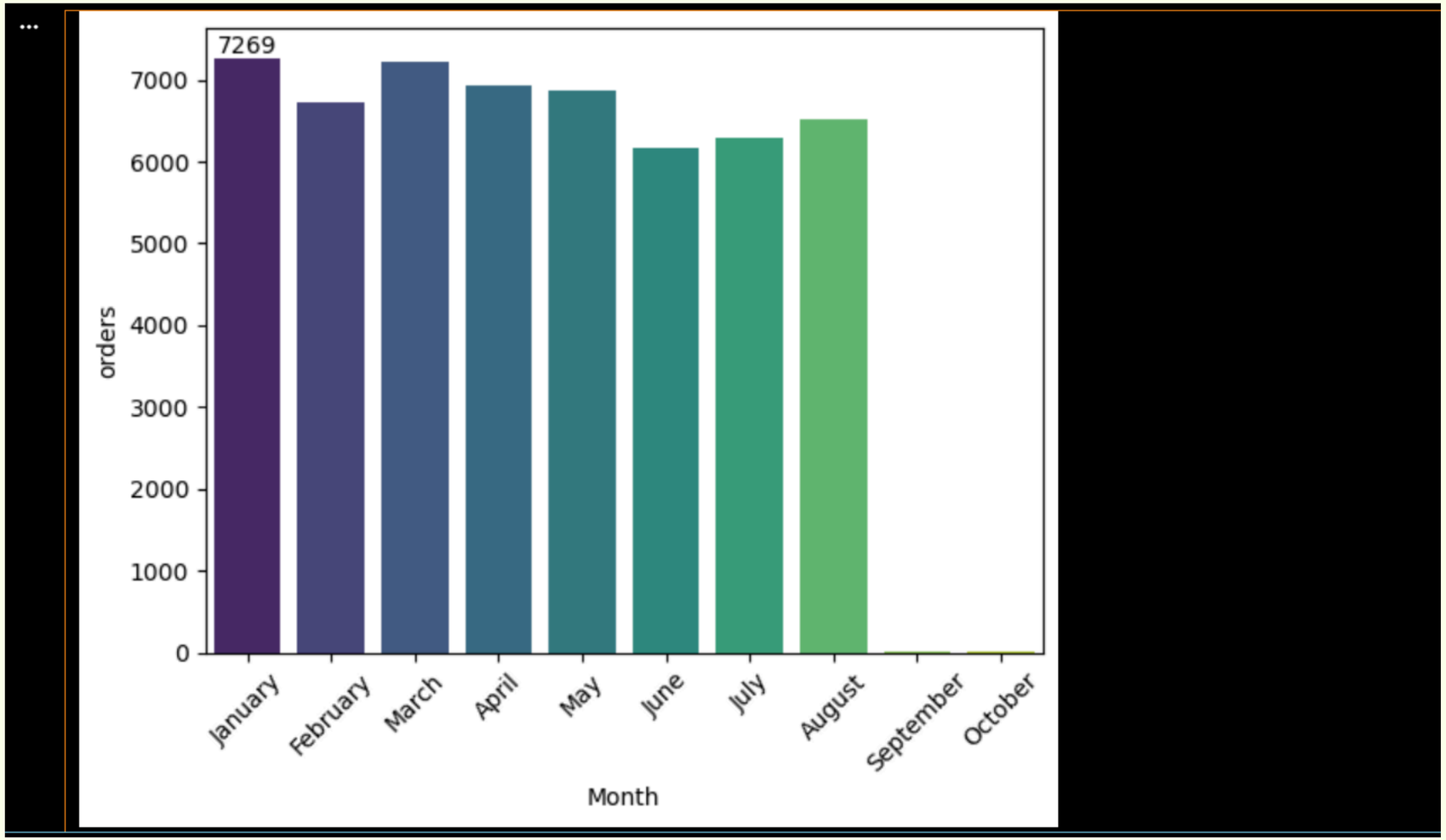
	category	sales
0	FURNITURE DECORATION	1430176.39
1	TELEPHONY	486882.05
2	BED TABLE BATH	1712553.67
3	AUTOMOTIVE	852294.33
4	COMPUTER ACCESSORIES	1585330.45
...	...	...
69	CDS MUSIC DVDS	1199.43

4. Calculate the percentage of orders that were paid in installments.

```
query = '''select (sum(case when payments.payment_installments >=1 then 1 else 0 end))/count(*)*100
    as orde_installments from payments'''
cur.execute(query)
data = cur.fetchall()
data[0][0]
```

Python

```
Decimal('99.9981')
```

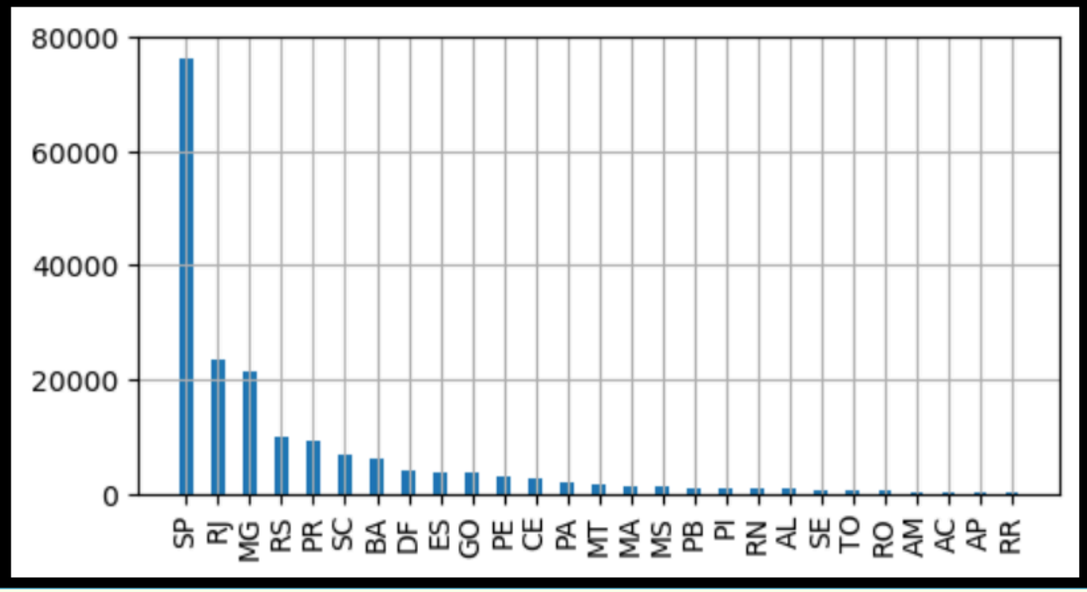

7269
7000
6000
5000
4000
3000
2000
1000
0
orders
January
February
March
April
May
June
July
August
September
October
Month

```
data = cur.fetchall()

df = pd.DataFrame(data, columns=["Customers", "State"])
df = df.sort_values(by = "Customers", ascending = False)
plt.figure(figsize=(6,3))
plt.bar(df["State"], df["Customers"], width=0.4)
plt.xticks(rotation=90)
plt.grid()

plt.show()
```

[6]



1. Calculate the number of orders per month in 2018.

```
query = '''select monthname(order_purchase_timestamp) as month_,count(orders.order_id) as order_count
        from orders where year(order_purchase_timestamp) = 2018 group by month_ order by month_'''
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data, columns=["Month", "orders"])
o = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October"]
bar = sns.barplot(x = df["Month"], y = df["orders"], data = df, order = o, palette = "viridis")
bar.bar_label(bar.containers[0])
plt.xticks(rotation= 45)
plt.show()
```

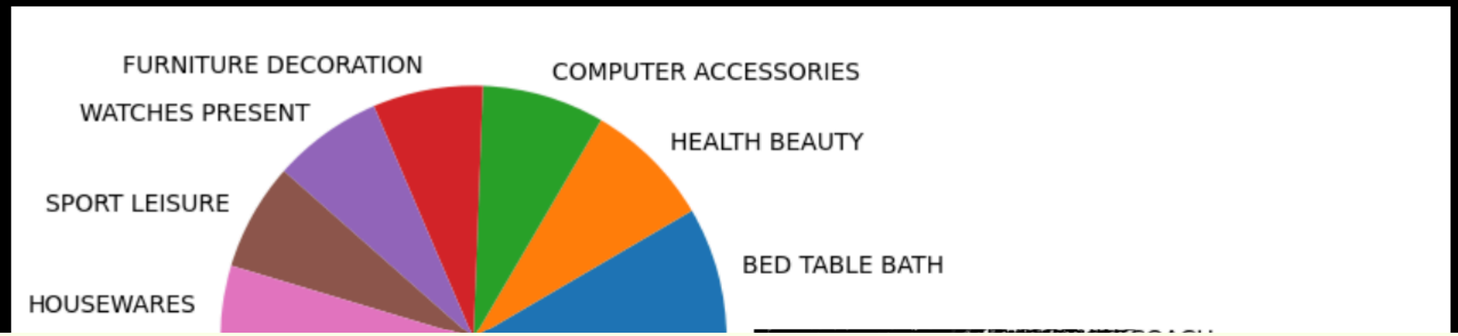
[7]

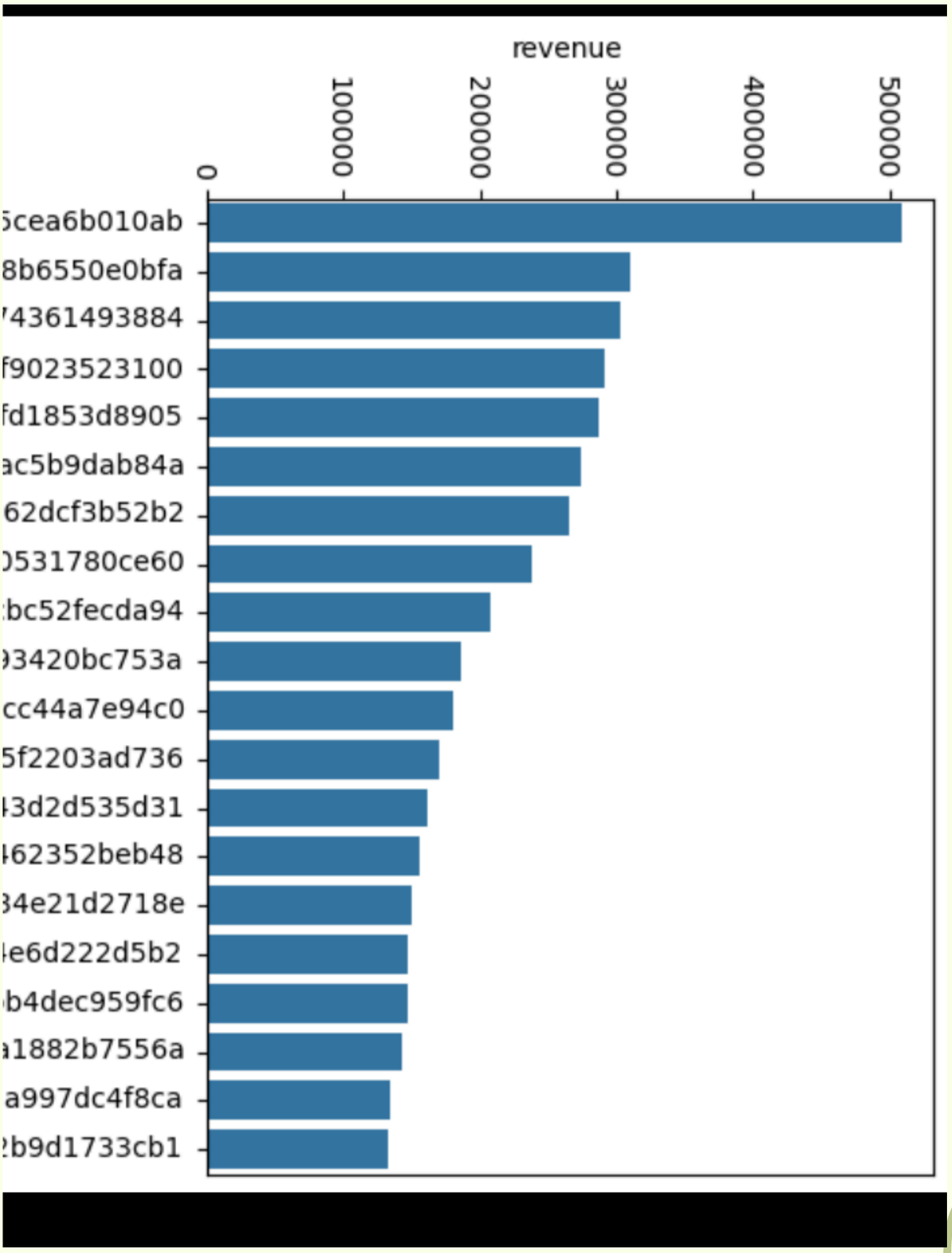
3. Calculate the percentage of total revenue contributed by each product category.

```
query = '''with category_sales as (select upper(products.product_category) as category, round(sum(payments.payment_value),2) as sales
         from products join orderitems on products.product_id = orderitems.product_id join payments on
         payments.order_id = orderitems.order_id group by category)
select category, round(sales/sum(sales) over()*100, 2) as percentage_of_rev from category_sales order by percentage_of_rev desc;'''
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data, columns = ["Category", "Percentage Distribution"])
df.head()
```

[6] Python

...




revenue
0
100000
200000
300000
400000
500000
5cea6b010ab
8b6550e0bfa
'4361493884
f9023523100
fd1853d8905
ac5b9dab84a
62dcf3b52b2
0531780ce60
bc52fecda94
93420bc753a
cc44a7e94c0
5f2203ad736
43d2d535d31
462352beb48
34e21d2718e
4e6d222d5b2
b4dec959fc6
a1882b7556a
a997dc4f8ca
2b9d1733cb1

1. Calculate the number of orders per month in 2018.

[7]

```
query = '''select monthname(order_purchase_timestamp) as month_,count(orders.order_id) as order_count
        from orders where year(order_purchase_timestamp) = 2018 group by month_ order by month_'''
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data, columns=["Month", "orders"])
o = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October"]
bar = sns.barplot(x = df["Month"], y = df["orders"], data = df, order = o, palette = "viridis")
bar.bar_label(bar.containers[0])
plt.xticks(rotation= 45)
plt.show()
```

5. Identify the top 3 customers who spent the most money in each year.

```
query = '''select years, customer_id, payment, d_rank
from
(select year(orders.order_purchase_timestamp) years, orders.customer_id, sum(payments.payment_value) payment,
dense_rank() over (partition by year(orders.order_purchase_timestamp) order by sum(payments.payment_value) desc) d_rank
from orders join payments on payments.order_id = orders.order_id
group by year(orders.order_purchase_timestamp), orders.customer_id)as a
where d_rank <= 3;'''
cur.execute(query)
data = cur.fetchall()
df = pd.DataFrame(data)
df.head()
```

	0	1	2	3
0	2016	a9dc96b027d1252bbac0a9b72d837fc6	1423.55	1
1	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1400.74	2
2	2016	4a06381959b6670756de02e07b83815f	1227.78	3

1. List all unique cities where customers are located.

[2]

```
query = f'select distinct customer_city from customers'
cur.execute(query)
data = cur.fetchall()
data
```

```
(('franca',),
 ('sao bernardo do campo',),
 ('sao paulo',),
 ('mogi das cruzes',),
 ('campinas',),
 ('jaragua do sul',),
 ('timoteo',),
 ('curitiba',),
 ('belo horizonte',),
 ('montes claros',),
 ('rio de janeiro',),
 ('lencois paulista',),
 ('caxias do sul',),
 ('piracicaba',),
 ('guarulhos',),
```